

Week 5: Spark - Data Cleaning, Transformation and Aggregation

Name: Manas Arora

Intern ID: CT_CSI_DE_1089

Objective Understand Spark fundamentals and perform data cleaning, transformation, and aggregation using DataFrames.

Dataset: Kaggle - Retail Analysis Large Dataset

Column mapping used below (assignment terms -> actual columns):

- user_id -> Customer_ID
- transaction_date -> Date
- region -> Country
- product_category -> Product_Category
- sale_amount -> Total_Amount
- price -> Amount
- subscription -> Customer_Segment
- status -> Order_Status
- store_id -> City
- username -> Name

```
from pyspark.sql import SparkSession
```

```
from pyspark.sql.functions import *
```

```
from pyspark.sql.types import *
```

```
spark = SparkSession.builder.appName("Week5_Spark_Assignment").getOrCreate()
```

```
print("Spark Session Created Successfully")
```

```
df = spark.read.csv("DataSet/retail_data.csv", header=True, inferSchema=True)
```

```
print("Data set readed")
```

```
df.show(5)
```

```
df.printSchema()
```

```
print("Total Rows:", df.count())
```

```
print("Total Columns:", len(df.columns))
```

```
df.describe().show()
```

Assignment Questions

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Limitations of MapReduce:

- Disk-based processing between every stage
- Slow for iterative jobs
- High latency, complex programming model
- No native support for streaming/ML

Spark advantages: in-memory computing, faster execution, easy DataFrame API.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

- Disk-based systems read/write data from disk in every iteration.
- Spark loads data once and keeps it in memory, so repeated steps (like ML training) are much faster.
- Less disk I/O means better performance for iterative algorithms.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

```
df_no_duplicates = df.dropDuplicates(["Customer_ID", "Date"])
```

```
print("Rows Before:", df.count())  
print("Rows After:", df_no_duplicates.count())  
df_no_duplicates.show(5)
```

Q4: Given a DataFrame df_sales, filter for region = 'West', then group by product_category to find the average sale_amount.

```
df_sales = df
```

region -> Country in this dataset, change 'USA' to a value present in your data

```
avg_sales = df_sales.filter(col("Country") == "USA") \  
    .groupBy("Product_Category") \  
    .agg(avg("Total_Amount").alias("avg_sale_amount"))
```

```
avg_sales.show()
```

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

- `.na.drop()` removes rows that have null values.
- `.na.fill()` fills the null values with a given value instead of removing the row.

```
df_status = df.na.fill({"Order_Status": "Unknown"})
df_status.select("Customer_ID", "Order_Status").show(5)
```

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

```
city_counts = df.groupBy("City").agg(count("*").alias("record_count"))
```

```
city_counts_filtered = city_counts.filter(col("record_count") > 100)
```

```
city_counts_filtered.show()
```

Q7: How does the immutability of Spark DataFrames affect how you perform data cleaning steps like dropping columns or renaming them?

- Spark DataFrames cannot be changed in place, every operation creates a new DataFrame.
- So we need to store the result in a variable, like `df = df.drop("column_name")`.
- The original DataFrame remains unchanged unless we overwrite it.

```
df_dropped = df.drop("Feedback")
df_renamed = df_dropped.withColumnRenamed("Total_Amount", "total_sale")
```

```
df_renamed.show(5)
```

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

```
df_filtered = df.filter((col("Age") >= 18) & (col("Age") <= 30) & (col("Customer_Segment") == "Premium"))
```

```
df_filtered.show(10)
```

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

- sum() and avg() skip null values automatically, which can give a wrong picture of the data.
- If we fill or drop nulls first, the aggregation result is more accurate and consistent.

Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

```
df_ts = df.withColumn("Date", to_timestamp(col("Date"), "M/d/yyyy"))
df_ts = df_ts.withColumnRenamed("Date", "event_time")
```

```
df_ts.select("Customer_ID", "event_time").show(5)
```

Q11: Explain the Shuffle process that occurs during a grouping operation. Why is it considered a wide transformation?

- Shuffle means moving data across different partitions/nodes so that same keys come together.
- groupBy needs all rows of the same key on one node, so Spark shuffles the data - this makes it a wide transformation, unlike filter/select which work on data already present in the partition.

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

```
clean_df = df.filter(col("Email").isNotNull())
clean_df = clean_df.filter(col("Name") != "")
```

```
clean_df.show(5)
```

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

```
df.agg(
    min("Amount").alias("min_price"),
    max("Amount").alias("max_price"),
    avg("Amount").alias("mean_price")
).show()
```

Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

- Spark may guess the wrong data type for the column.
- Some dates may not match the guessed format and become null.
- This can lead to wrong analysis later.

Q15: Write a final processing pipeline that: 1. Filters out duplicates. 2. Fills null prices with 0. 3. Groups by store_id to calculate total revenue.

```
final_df = df.dropDuplicates()
final_df = final_df.na.fill({"Amount": 0})

revenue = final_df.groupBy("City").agg(sum("Amount").alias("total_revenue"))

revenue.show()

# Save cleaned dataset
cleaned_df = df.dropDuplicates()
cleaned_df = cleaned_df.na.fill({"Amount": 0})

cleaned_df.toPandas().to_csv("cleaned_dataset.csv", index=False)
print("Cleaned dataset saved successfully.")
```